

Encontro 4: Laços de Repetição, Funções e Lógica Aplicada

Sumário

Encontro 4: Laços de Repetição, Funções e Lógica Aplicada.....	1
1. Introdução aos Laços de Repetição.....	2
2. O Laço `while` (Enquanto).....	2
3. O Laço `for` (Para).....	2
4. Modularização: Funções e Procedimentos.....	3
5. Algoritmos e Lógica Aplicada.....	4
6. Exercícios Práticos	5
6.1. Exercício 1: Somatório com `while`	5
6.2. Exercício 2: Fatorial com `for`	5
6.3. Exercício 3: Função de Verificação de Número Primo	6

1. Introdução aos Laços de Repetição

Em programação, muitas vezes precisamos executar o mesmo bloco de código várias vezes. Em vez de repetir o código manualmente, usamos **laços de repetição** (ou loops). Eles permitem que uma sequência de instruções seja executada repetidamente até que uma determinada condição seja satisfeita. Neste encontro, vamos explorar os laços `while` e `for`, que são fundamentais para automatizar tarefas repetitivas.

2. O Laço `while` (Enquanto)

O laço `while` executa um bloco de código **enquanto** uma condição especificada for verdadeira. É ideal para situações em que não sabemos de antemão quantas vezes o loop precisa ser executado.

Sintaxe em Pseudocódigo:

```
enquanto (condição) faça  
    // Bloco de código a ser repetido  
fimenquanto
```

Exemplo: Contagem até 5

```
start  
declare counter: integer = 1  
while (counter <= 5) do  
    write counter  
    counter = counter + 1  
endwhile  
end
```

Figura 1: Exemplo de laço while em pseudocódigo.

A figura 1 demonstra como usar uma variável de controle (contador) para repetir um bloco de código enquanto uma condição for verdadeira. É fundamental incrementar a variável dentro do loop para evitar loops infinitos.

Neste exemplo, o loop continuará executando enquanto a variável `contador` for menor ou igual a 5. A cada iteração, o valor do contador é exibido e, em seguida, incrementado. É crucial que a variável de controle (`contador`) seja modificada dentro do loop para evitar um **loop infinito**.

3. O Laço `for` (Para)

O laço `for` é geralmente usado quando sabemos o número exato de vezes que queremos executar um loop. Ele combina a inicialização da variável de controle, a condição de continuação e o incremento em uma única linha, tornando o código mais compacto.

Sintaxe em Pseudocódigo:

```
para (inicialização; condição; incremento)
    // Bloco de código a ser repetido
fimpara
```

Exemplo: Tabuada do 7

```
start
|
declare i: integer
for i from 1 to 10 do
    write
        7 x, i, =, (7 * i)
    endfor
|
end
```

Figura 2: Exemplo de laço for para gerar tabuada em pseudocódigo.

A figura 2 mostra como o laço for é mais compacto para situações nas quais sabemos o número exato de iterações, combinando inicialização, condição e incremento em uma estrutura única.

Este loop executará 10 vezes, com *i* assumindo os valores de 1 a 10. Em cada iteração, ele calcula e exibe a multiplicação de 7 pelo valor atual de *i*.

4. Modularização: Funções e Procedimentos

À medida que nossos programas crescem, é essencial organizá-los em partes menores e reutilizáveis. A **modularização** é a técnica de dividir um programa grande em módulos menores e independentes. **Funções** e **procedimentos** são os blocos de construção da modularização.

- **Procedimento:** Um bloco de código que realiza uma tarefa específica. Ele pode receber parâmetros, mas não retorna um valor.
- **Função:** Semelhante a um procedimento, mas com uma diferença crucial: ela **retorna um valor** ao final de sua execução.

Sintaxe de uma Função em Pseudocódigo:

```
funcao nome_da_funcao(parametro1, parametro2) : tipo_de_retorno
    // Bloco de código
    retorne valor
fimfuncao
```

Exemplo: Função para Somar

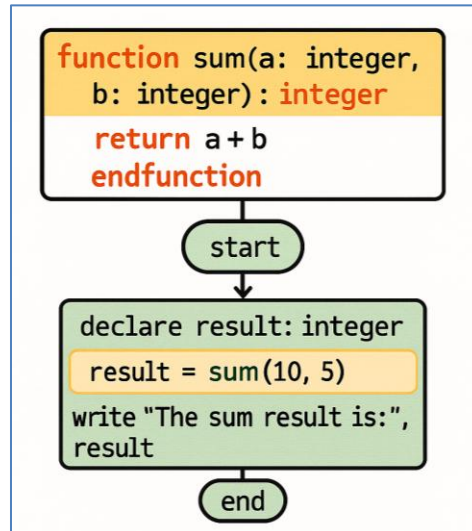


Figura 3: Exemplo de função em pseudocódigo.

A figura 3 demonstra a definição de uma função que recebe parâmetros, executa uma operação e retorna um valor. Mostra também como chamar a função no programa principal e usar o valor retornado.

O uso de funções torna o código mais legível, mais fácil de depurar e permite o reaproveitamento de código, seguindo o princípio **DRY (Don't Repeat Yourself - Não se Repita)**.

5. Algoritmos e Lógica Aplicada

Agora que conhecemos as estruturas de controle e a modularização, podemos começar a construir algoritmos mais complexos para resolver problemas do mundo real. A chave para a lógica aplicada é a capacidade de **decompor um problema** em etapas menores e, em seguida, usar as ferramentas de programação que aprendemos para implementar a solução.

Passos para a Resolução de Problemas:

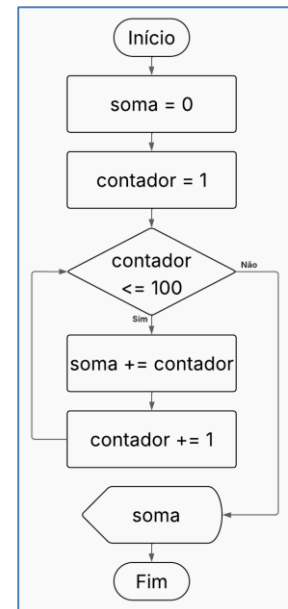
1. **Entender o Problema:** Leia e releia o enunciado. Quais são as entradas? Qual é a saída esperada? Quais são as regras e restrições?
2. **Planejar a Solução:** Pense em um plano passo a passo (o algoritmo). Você pode usar um fluxograma ou pseudocódigo para visualizar a lógica.
3. **Codificar a Solução:** Traduza seu algoritmo para a linguagem de programação (ou pseudocódigo).
4. **Testar a Solução:** Teste seu programa com diferentes entradas, incluindo casos extremos (ex: números negativos, zero, valores muito grandes) para garantir que ele funcione corretamente em todas as situações.

6. Exercícios Práticos

6.1. Exercício 1: Somatório com `while`

Crie um algoritmo que use um laço `while` para calcular a soma de todos os números de 1 a 100.

Este exercício demonstra como usar uma variável acumuladora (`soma`) junto com uma variável de controle (`contador`) para processar uma sequência de números.

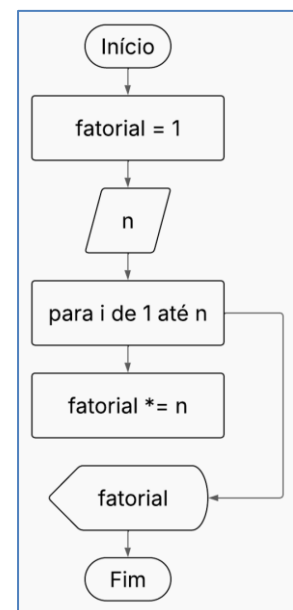


6.2. Exercício 2: Fatorial com `for`

Desenvolva um algoritmo que leia um número inteiro e calcule o seu fatorial usando um laço `for`.

O fatorial de um número n (representado por $n!$) é o produto de todos os inteiros positivos de 1 a n . (Ex: $5! = 5 * 4 * 3 * 2 * 1 = 120$).

A figura mostra como usar uma variável acumuladora (`fatorial`) que é multiplicada pelos valores sequenciais para obter o resultado final.



6.3. Exercício 3: Função de Verificação de Número Primo

Crie uma função que receba um número inteiro como parâmetro e retorne `verdadeiro` se o número for primo e `falso` caso contrário.

Um número primo é aquele que é divisível apenas por 1 e por ele mesmo.

```
function isPrime(number: integer): boolean
    if (number <= 1) then
        return false
    for i from 2 to sqrt(number) do
        if (number % i == 0) then
            return false
        endfor
    return true
endfunction
```

Este exercício demonstra o uso de estruturas de controle dentro de uma função, incluindo condições de parada antecipada e otimização do algoritmo testando apenas até a raiz quadrada do número.